Patti Royale — an in-game commerce store

Product Associate assignment · PlaySuper

Karan Makol · karan.makol1@gmail.com · August 2026

Live prototype: https://daenerysdrag.github.io/patti-royale-playsuper/
Code: https://github.com/DaenerysDrag/patti-royale-playsuper

Best viewed on a laptop — the PM console docks on the right and the reasoning callouts sit beside the phone. Works on mobile too.

Thesis

An in-game store that behaves like a shop will fail, because the player did not come to shop. There is no purchase intent to capture. The job is to convert accumulated play into a reason to play again.

> **In e-commerce, the cart is dead intent. In a game, the cart is a retention loop.**

So I did not build a storefront. I built a coin sink that lifts retention without touching IAP.

Assumptions

	WHAT I ASSUMED
Game	*Patti Royale* — casual trick-taking cards. 1M DAU, India, Android-first. 1.8 sessions/day, 4 matches/session. Baseline ARPDAU **₹0.80**
Player	Zero purchase intent on entry. ~40 seconds of attention between matches. Starts from the assumption that in-game "free rewards" are a scam
Archetypes	**Grinder** (daily, never pays, 528 coins/day) · **Dipper** (2–3×/week, churn-risk, 61/day) · **Whale** (buys IAP, 235/day)
Currency	Win 50 · loss 15 · daily streak 40. Peg: **10 coins = ₹1**
Non-goals	Does not touch the existing cosmetic coin sink. Does not touch IAP pricing. Coins never buy in-game power
Who pays	🔑 The **brand** funds the discount as customer-acquisition cost — not the studio

The consequence of that last row

If the brand funds the discount as CAC, then the constraint on this economy is **brand CAC budget and coupon inventory — not coin supply.**

Coins can be generous. What must be rationed is **conversion**. That is why there is a redemption cap (2/week) and no paywall on earning — and it is why I never modelled coins as a studio cost.

Tools used

TOOL	WHAT FOR
Claude Code (Opus 5)	The build — architecture, every screen, the trick-taking match logic, the event bus, the economy derivation, these docs
Claude Design	A parallel visual pass: I designed the store as a multi-screen canvas, then ported its typography, button geometry and instrumentation panel into the working build

TOOL	WHAT FOR
React 19 · TypeScript · Vite · Tailwind 4 · framer-motion · zustand	Stack. No backend; state persists to `localStorage`
GitHub Pages + Actions	Deploy. Every push to `main` typechecks, builds and republishes

How I used them matters more than that I used them.

I wrote a **7-phase gated brief** for Claude Code first and made it stop at checkpoints for review, so the decisions stayed mine. Twice it flagged that my own numbers broke — the original earn rate made a "this month" reward affordable in five days, and a single set of price bands cannot be "reach today" for both a Grinder and a Dipper. Both flags changed the design. The economy here is the second version, not the first.

The Claude Design pass was deliberately **not** adopted wholesale, and what I rejected from it is the more useful thing to know. I took its type scale, its pill button geometry, and its instrumentation-panel treatment, which were all better than my first pass. I left behind its light iris-and-cyan palette — a white SaaS surface bolted to a dark felt card table would read as two products — and I left behind its economy readout panel, because that design's own footnote said its funnel and economy figures were *"illustrative, not instrumented."* This build's console can't carry a section it would have to fake.

The core question: how should this differ from e-commerce?

E-COMMERCE ASSUMES	IN-GAME REALITY	SO I BUILT
The user arrives **with intent**	The player came to play; commerce interrupts	The store opens at the **post-match moment**, never as a nav tab
Search is the discovery primitive	40-second window; choice paralysis kills it	**No search, no filters.** 12 items, 3 fixed groups
Price is an **economic fact**	Coins are printed by the studio and cost the player *time*	Price shown in **effort** — "3 more wins" — computed from your earn rate
Not buying is **neutral**	"You can't afford this" demotivates — damaging the retention this was sold to fix	**Zero dead ends.** Every locked item shows a ring and a Play button
Infinite catalog wins	Infinite catalog dilutes	12 SKUs, digital only
A **cart** for multi-item baskets	Single-item impulse only	No cart. Two taps
Currency is **fungible cash**	A currency that buys real goods will cannibalise IAP	Real-world rewards only. **Never in-game power**
The store is **the destination**	Every second in-store costs the studio the session	Exit CTA is **"Back to match"**, never "Continue shopping"
Returns handle regret	You cannot refund someone's *time*	**Coins refunded if a voucher expires unused.** No e-commerce analogue exists
Growth = **more traffic**	Growth = **more reasons to play**	**Goals** — below

Goals — the mechanic the whole thing rests on

Reserve a reward and it becomes a **ring in the game HUD**, price-locked for 7 days. Every hand fills it. At 100% the game tells you.

This inverts the funnel from `browse → buy` into `want → play → afford → buy`. Commerce stops being an interruption and becomes the reason for the next session. It is also the only part of this design that plausibly moves D7 retention — which is what a studio is actually buying.

Two deliberate consequences:

- `goal_created` **is the conversion event that matters**, not `payment_success`. Creating a goal costs the player nothing and costs the brand nothing, and it is what brings them back.
- **A player holding no goal gets one manufactured.** The reward screen surfaces the nearest out-of-reach item rather than a catalog: close enough to want, far enough to play for.

The economy, and one derivation worth your time

Coin cap is a property of the **item**, never of the player: Tier 1 and 2 are fully coin-funded, **Tier 3 caps coins at 40% and the rest is cash** — PlaySuper's model, and the only place the coin↔cash slider appears. One flat rule, identical for everyone. I deliberately rejected loyalty tiers and engagement-based pricing: charging a more-engaged player more for the same item is manipulative, and it reads that way the moment two players compare screenshots.

Back-solving your published +5.2% ARPDAU rather than guessing a redemption rate:

```
₹0.80 baseline × 5.2% lift          = ₹0.0416 incremental per DAU per day
₹700 order × 10% commission × 50%   = ₹35 studio revenue per redemption
──────────────────────────────────────────────────────────────────────
⇒ 0.0416 / 35 = 0.119% of DAU must redeem daily
⇒ ≈1,189 redemptions/day at 1M DAU  ⇒ ≈₹5.35M/month of brand-funded discount
```

Only about 1 player in 840 per day. That is a tiny conversion requirement, and it changes the product: optimise for **retention breadth** — many players holding a goal — over **conversion depth**. A held goal costs the brand nothing and returns a player tomorrow; a checkout costs CAC and may not.

It is also why the conversion event I optimise for is **reserving a goal**, not completing a payment. Full funnel and event spec in `docs/03-event-taxonomy.md`.

(₹700 AOV, 10% commission, 50% studio share are my assumptions. If commission is 5%, the required redemption rate doubles and the catalog has to get cheaper — that's the first number I'd want from you.)

One thing I decided not to fake. A delivered coupon is not brand revenue — the brand's real conversion happens off-platform, days later, at the merchant's checkout. So the prototype's console shows the redemption it can observe and greys out the one it cannot, rather than inventing a figure. Closing that gap needs unique per-player codes and a brand-side redemption webhook. Until it exists, every ROI number quoted to a brand is a proxy and should be labelled as one.